

Transport Layer & Congestion Control - Detailed Notes

■ Nagle's Algorithm

Purpose: To reduce the number of small packets sent over the network (called tinygrams).

How it works:

- When data is small, TCP waits until either:
 1. The previous packet is acknowledged, or
 2. Enough data is collected to send a full segment.

Example: Combines small keystrokes into one segment.

Advantage: Reduces congestion.

Disadvantage: Increases delay in interactive applications (e.g., gaming, telnet).

■ Network Layer Congestion

1. **Throughput:** Rate of successful data delivery. High congestion = lower throughput.
2. **Delay:** Time taken for data to travel. High congestion = higher delay.

■■ Congestion

Occurs when the network is overloaded — too many packets compete for limited resources.

Causes of Congestion:

1. High data rate
2. Insufficient buffer
3. Low bandwidth links
4. Retransmissions
5. Bursty traffic

■ Difference between Congestion Control and Flow Control

Aspect | Congestion Control | Flow Control

Purpose | Prevent network congestion | Prevent receiver overflow

Scope | Entire network | Between sender and receiver

Managed by | Network and transport layers | Transport layer (TCP)

Example | TCP slow start | TCP sliding window

■■ Principle of Congestion Control

- Monitor network load
- Adjust sending rate dynamically
 - If congestion detected → reduce rate
 - If network is free → increase rate

TCP uses:

- Slow Start
- Congestion Avoidance
- Fast Retransmit
- Fast Recovery

■ Congestion Prevention Policies

1. Retransmission Policy – Avoid unnecessary retransmission.
2. Window Policy – Adjust sender's window size.
3. Acknowledgment Policy – Use delayed ACKs.
4. Discarding Policy – Drop low-priority packets.
5. Admission Policy – Limit new connections.

■ TCP Congestion Control - Slow Start Algorithm

- Begins with small congestion window ($cwnd = 1 \text{ MSS}$).
- Increases exponentially ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$).
- Stops increasing when threshold ($ssthresh$) is reached.

Congestion Avoidance (Additive Increase)

After threshold, $cwnd$ increases linearly ($cwnd + 1 \text{ MSS per RTT}$).

Prevents network overload.

Example: $16 \rightarrow 17 \rightarrow 18 \rightarrow 19 \rightarrow 20$.

■■ TCP Timers

1. Retransmission Timer
2. Persistence Timer
3. Keepalive Timer
4. TIME-WAIT Timer

■ Other TCP Options

1. End of Option (EOP) – Marks end of header options.
2. MSS – Maximum segment size (e.g., 1460 bytes).
3. Window Scale – For larger windows.
4. Timestamp – Measures RTT accurately.
5. SACK – Selective retransmission of missing segments.

■■ Comparison: UDP vs TCP

Feature | UDP | TCP
Type | Connectionless | Connection-oriented
Reliability | Unreliable | Reliable
Flow Control | None | Yes
Congestion Control | None | Yes
Speed | Fast | Slower
Example | Video streaming, DNS | File transfer, Email

■ Stream Control Transmission Protocol (SCTP)

A message-oriented, reliable, transport layer protocol designed to overcome TCP/UDP limitations.

■ UDP Performance for Internet Applications

- Low delay due to no connection setup.
- Ideal for real-time apps (VoIP, video calls).
- No retransmission overhead.

■ TCP Performance for Internet Applications

- Reliable but may suffer latency.
- Great for file transfer, email, web browsing.

■ SCTP Performance for Internet Applications

- Combines best of both TCP and UDP.
- Reliable, faster recovery, multi-streaming.
- Ideal for signaling, multimedia, telecom.

■ SCTP Services

1. Process-to-process communication
2. Multiple streams
→ Prevents head-of-line blocking
3. Multihoming
4. Full-duplex communication
5. Connection-oriented
6. Reliable service

■ SCTP Features

Feature | Description
Transmission Sequence Number | Ensures reliable delivery
Stream Identifier | Identifies streams

Stream Sequence Number | Maintains order

Packets | Contain data chunks

Acknowledgment Number | Confirms data

Flow Control | Manages data rate

Error Control | Detects & retransmits

Congestion Control | Similar to TCP